Data Cleaning Mastery Lab

Objective

In this lab, you will take a messy, real-world dataset and apply professional data cleaning techniques. This exercise will test your ability to profile data, handle missing values, format data types, manage outliers, and prepare a dataset for machine learning or visualization.

Part 1: Dataset Selection and Import

Choose **ONE** of the following datasets to work on. You will need to create a free Kaggle account to download them.

- **Option A (Recommended): New York City Airbnb Open Data**
    - *Link:* https://www.kaggle.com/datasets/dgomonov/new-york-city-airbnb-open-data
    - *Description:* This dataset contains details about Airbnb listings, hosts, locations, and reviews. It is famous for having missing dates, varied text formats, extreme price outliers, and unneeded columns.
- **Option B: Customer Personality Analysis**
    - *Link:* https://www.kaggle.com/datasets/imakash3011/customer-personality-analysis
    - *Description:* A detailed analysis of a company's ideal customers. It requires heavy cleaning of dates, handling nulls in income, and creating new categorical variables.

Part 1:

1. Download your chosen dataset CSV file.
2. Create a new Jupyter Notebook or Python script.
3. Import the necessary libraries.
4. Load the CSV file into a pandas DataFrame.

Part 2: Data Profiling & Initial Inspection

Before changing anything, you must understand the current state of your data.

Part 2:

1. Display the first 10 rows and the last 5 rows of the dataset.
2. Print the shape of the DataFrame to see how many rows and columns you are starting with.
3. Generate a summary of the DataFrame to inspect the data types of each column and see the count of non-null values.
4. Generate descriptive statistics for all numerical columns. Take note of the `min` and `max` values—do any numbers look suspiciously high or low? (e.g., an Airbnb price of $0).
5. Calculate and print the exact total number of missing (null) values for *every single column* in the dataset.

Part 3: Structural Cleaning

Remove data that will not serve your future analysis to save memory and reduce clutter. *(Note: The column names below refer to the Airbnb dataset. If you chose Option B, adapt these steps to irrelevant columns like 'Z_CostContact').*

Part 3:

1. Drop the columns that are unnecessary for behavioral or geographic analysis. For the Airbnb dataset, drop the `id` and `host_name` columns. (Dropping `host_name` also helps anonymize the data).
2. Rename the column `neighbourhood_group` to `borough`.
3. Standardize all remaining column names by ensuring they are completely lowercase and contain no spaces (replace any spaces with underscores if they exist).

Part 4: Handling Missing Data

Dealing with missing values requires domain knowledge and strategic choices.

Part 4:

1. **Categorical/Text Missing Data:** Find the column representing the name of the listing. Fill any missing values in this column with the string `"Unknown"`.
2. **Numerical Missing Data:** Find the column representing the number of reviews per month. It likely has missing values because some properties have never been reviewed. Fill all missing values in this column with the number `0`.

3. **Date Missing Data:** Find the column representing the date of the last review. We cannot accurately guess a missing date. Leave the nulls here for now, but note how many there are.

4. Run your missing value check from Part 2 again to prove that the 'name' and 'reviews per month' columns now have zero missing values.

Part 5: Data Formatting and Type Conversion

Data must be in the correct format (datetime, string, category, integer) for analysis algorithms to interpret them correctly.

Part 5:

1. **Datetime Conversion:** The `last_review` column is currently being read as a standard text/object string. Convert this entire column into proper pandas datetime objects.

2. **Categorical Optimization:** Look at the `room_type` and `borough` columns. These contain only a few unique text values repeated thousands of times. Convert both of these columns from 'object' data types to 'category' data types to save memory.

3. Verify your changes by printing the data types of the DataFrame again.

Part 6: Outliers and Duplicates

Real data is messy and prone to human error during entry.

Part 6:

1. **Duplicates:** Check for any completely identical rows in the dataset. If any exist, drop them, keeping only the first occurrence.

2. **Nonsense Values:** Find any rows where the `price` is exactly 0. An Airbnb cannot be free. Filter the DataFrame to remove any rows where the price is 0.

3. **Extreme Outliers:** The `price` column has extreme upper outliers (e.g., $10,000 a night). Calculate the 99th percentile of the `price` column. Filter your DataFrame to keep *only* the rows where the price is less than this 99th percentile value.

4. Print the shape of your DataFrame. Compare it to your original shape from Part 2 to see how many rows were removed.

Part 7: Feature Engineering

Create new data points from existing ones to make future analysis easier.

Part 7:

1. **Binning:** We want to classify properties by affordability. Create a new column called `price_tier`.
2. Use a pandas binning method to categorize the `price` column into three buckets:
    - `Budget`: $1 to $100
    - `Mid-Range`: $101 to $250
    - `Luxury`: $251 and above
3. Check your work by printing the value counts of your new `price_tier` column.

Part 8: Final Export

Save your hard work so it can be passed on to the data visualization or machine learning team.

Part 8:

1. Export your fully cleaned DataFrame to a new CSV file.
2. Name the file `cleaned_dataset_v1.csv`.
3. Ensure that you do *not* export the DataFrame's index column into the new CSV (you will need to specify a parameter to prevent this).